Software Requirement Specification document - MedGuardAI

📘 SECTION 1 – INTRODUCTION

1.1 Purpose of the System

MedGuard AI is an **Intelligent Medicine Verification Platform** designed to authenticate medicines, detect expiry, validate legality (India-focused), and provide AI-generated summaries using a multi-modal and microservice-based architecture.The system also includes a separate **LLM-based Symptom Remedy Module** that generates safe home remedies based on user-entered symptoms.

The purpose of this platform is to:

- Improve patient safety
- Reduce counterfeit medicine circulation
- Provide accessible verification tools
- Offer safe informational home remedies
- Build a scalable healthcare AI infrastructure

1.2 Problem Statement

The pharmaceutical market faces increasing issues of:

- Counterfeit medicines
- Expired products still in circulation
- Illegal or unregistered drugs
- Lack of consumer awareness
- Slow manual verification processes

Existing verification systems:

- Are manual
- Require expert knowledge
- Are not consumer-friendly
- Do not support multi-modal inputs

There is a strong need for an:

Automated, scalable, AI-driven medicine verification platform accessible to the general public.

1.3 Scope of the System

MedGuard AI consists of two primary modules:

🔹 1.3.1 Medicine Verification Module

This module:

- Accepts text, image, or barcode input
- Extracts medicine information using OCR and barcode decoding

- Uses RAG + FAISS for fast semantic medicine matching
- Validates:
    - Authenticity
    - Expiry
    - Legality in India
- Generates structured AI summary

Target Users:

- Consumers
- Medical stores
- Pharmacies
- Healthcare professionals

🔹 **1.3.2 Symptom-Based Home Remedy Module**

This module:

- Accepts symptom text
- Uses a Local LLM (Ollama)
- Generates:
    - Safe home remedies
    - Safe proportions
    - Precaution notes
    - Mandatory disclaimer

Important Constraints:

- No diagnosis
- No prescription medicines
- No clinical treatment plans
- Informational use only

1.4 Objectives

The main objectives of MedGuard AI are:

1. Detect counterfeit medicines
2. Detect expired medicines
3. Validate legality in Indian context
4. Provide fast (<5 sec target) medicine search using FAISS
5. Generate AI-powered readable summaries
6. Provide safe home remedies via local LLM
7. Ensure privacy via local model execution
8. Build scalable Dockerized microservice architecture

1.5 System Overview

MedGuard AI follows a:

- Microservice Architecture

- Dockerized Deployment
- REST-based Inter-Service Communication
- RAG-based Semantic Retrieval System
- Local LLM Integration

The system is designed to be:

- Scalable
- Modular
- Fault-tolerant
- Cloud-deployable

📘 SECTION 2 – SYSTEM ARCHITECTURE (DETAILED VERSION)

2.1 Architectural Style

Architecture Pattern:

Dockerized Microservice Architecture with REST Communication and RAG-based Retrieval Layer.

Why this architecture:

- Independent scaling
- Separation of AI logic
- Fault isolation
- Easy deployment
- Future extensibility

2.2 High-Level Components

The system consists of the following services:

🔹 2.2.1 API Gateway Service

Technology:

- Django
- Django REST Framework
- JWT Authentication

Responsibilities:

- Acts as central entry point
- Handles authentication
- Validates requests
- Routes requests to appropriate microservice
- Logs requests
- Rate limiting (future enhancement)

Container Name:
api-gateway

🔹 2.2.2 Medicine Verification Service

Core responsibilities:

1. OCR Processing
2. Barcode Decoding
3. Data Extraction
4. RAG-based Search
5. Authenticity Verification
6. Expiry Check
7. Legality Check
8. Summary Generation

Technologies:

- Google Vision API (OCR)
- pyzbar (Barcode)
- Regex (MFG/EXP extraction)
- Sentence Transformers (Embedding generation)
- FAISS (Vector Search)
- Ollama (Summary LLM)

Container:
medicine-service

🔹 RAG + FAISS Workflow (Important for Viva)

1. Medicine dataset converted into embeddings
2. Stored in FAISS index
3. User input converted into embedding
4. FAISS similarity search
5. Top-K matches retrieved
6. Verification logic applied

Advantages:

- Semantic matching
- Very fast search
- Scalable to large datasets
- Better than fuzzy matching

🔹 2.2.3 Symptom Remedy LLM Service

Purpose:

- Generate safe home remedies
- Include proportions
- Add precaution notes
- Add disclaimer

Technology:

- Ollama
- Local LLM (Model TBD)
- Strict system prompt

Container:
symptom-llm-service

Why separate service?

- Fault isolation
- Independent scaling
- Clean separation of medical information logic
- Better microservice design

🔹 **2.2.4 Database Service**

Database:
PostgreSQL

Responsibilities:

- Store users
- Store medicines
- Store verification history
- Store embedding metadata

Container:
postgres-db

🔹 **2.2.5 FAISS Persistent Volume**

FAISS index stored in:

Docker-mounted volume

Benefits:

- Survives container restart
- No re-indexing needed
- Production-grade setup

🔹 **2.3 Service Communication Flow**

All services communicate using: REST APIs over Docker network.

Example Flow: User ——-> React / Flutter ——-> API Gateway ——-> REST call ——-> Medicine Service ——-> FAISS Search ——-> LLM Summary ——-> Return Response

🔹 **2.4 Deployment Architecture**

Deployment Strategy:

- Docker
- Docker Compose (Development)
- Cloud-ready (AWS / DigitalOcean)

Services:

- api-gateway
- medicine-service
- symptom-llm-service
- postgres-db
- ollama

🔹 2.5 Design Strengths

1. Microservice isolation
2. RAG-based intelligent retrieval
3. Local LLM (privacy-preserving)
4. Dockerized deployment
5. Fast vector search
6. Modular expansion capability

📊 SECTION 3 – DATABASE DESIGN (Conceptual & Logical Level – PostgreSQL)

Database Used: **PostgreSQL**

Design Approach:

- Relational database
- Third Normal Form (3NF)
- UUID as primary keys
- ENUM types for controlled values
- Indexed columns for performance
- Designed for microservice compatibility

🔹 3.1 Database Overview

The database is divided into five major modules:

1. User Management
2. Medicine Dataset
3. Barcode Management
4. RAG Embedding Metadata
5. Verification & Symptom History

Each module is logically separated but relationally connected using foreign keys.

🧑‍💻 3.2 User Management Module****Table: Users

Purpose:

Stores registered users of the platform.

Main Attributes:

- User ID (Primary Key – UUID)
- Name
- Email (Unique)
- Password (Hashed)
- Role (User / Admin / Pharmacy – future scalable)
- Created Timestamp
- Updated Timestamp

Key Design Decisions:

- Email is unique for authentication.
- UUID allows distributed system compatibility.
- Role field supports future admin dashboard.

💊 3.3 Medicine Data Module

This is the core table of the system.

Table: Medicines

Purpose:
Stores structured information about medicines.

Main Attributes:

- Medicine ID (Primary Key)
- Name
- Brand
- Manufacturer
- Composition
- Description
- Legality Status
- Created Timestamp

Legality Status Design

Instead of simple True/False, we use ENUM type:

- Legal
- Banned
- Restricted
- Under Review

Why ENUM?

- Prevents invalid entries
- Ensures regulatory clarity
- Stronger academically

🔎 3.4 Barcode Management Module
We separated barcodes into a different table.

Table: Medicine Barcodes

Purpose:
Allows one medicine to have multiple barcodes.

Attributes:

- Barcode ID
- Medicine ID (Foreign Key)
- Barcode Value (Unique)
- Barcode Type (EAN/UPC)
- Created Timestamp

Why Separate Table?

- One medicine can have multiple packaging variations.
- Industry-standard scalable design.
- Avoids redundancy.

🔍 3.5 RAG Embedding Metadata Module
Since we are using FAISS (external vector store), we must store mapping metadata in PostgreSQL.

Table: Embeddings Metadata

Purpose:
Stores mapping between medicine records and vector embeddings.

Attributes:

- Embedding ID
- Medicine ID (Foreign Key)
- Text Chunk Used for Embedding
- Embedding Model Version
- Created Timestamp

Why Needed?

- Enables re-indexing
- Supports model upgrades
- Keeps track of embedding versions
- Ensures traceability

🗒️ 3.6 Verification History Module****Table: Verification History

Purpose:
Stores all medicine verification attempts by users.

Attributes:

- History ID
- User ID (Foreign Key)
- Medicine ID (Foreign Key – nullable if not found)
- Input Type (Text/Image/Barcode)
- Raw Input Text
- Similarity Score (FAISS score)
- Result Status
- Checked Timestamp

Result Status ENUM

- Authentic
- Fake
- Expired
- Illegal
- Not Found

Why ENUM?

- Maintains strict control
- Prevents inconsistent values
- Makes analytics easier

🌿 3.7 Symptom Remedy Module (Optional Logging)**Table: Symptom History**

Purpose:
Stores user symptom inputs and generated remedies.

Attributes:

- Entry ID
- User ID
- Symptom Text
- LLM Response
- Timestamp

This table can be:

- Fully enabled
- Anonymized
- Disabled (privacy-first mode)

🔹 3.8 Entity Relationship Structure (Conceptual)

Users ——-> Verification History ——-> Medicines ——-> Medicine Barcodes ——-> Embeddings Metadata ——-> Symptom History is linked separately to Users.

🔹 3.9 Normalization Level

The database follows:

- First Normal Form (Atomic values)

- Second Normal Form (No partial dependencies)
- Third Normal Form (No transitive dependency)

No data duplication.

🔹 3.10 Performance Strategy

Performance optimization is achieved using:

- Indexed email for login
- Indexed barcode for instant match
- Indexed medicine name
- Indexed legality status
- Indexed timestamps for analytics
- FAISS for semantic search (outside SQL)

PostgreSQL handles structured queries.
FAISS handles vector similarity.

📘 SECTION 4 – FUNCTIONAL REQUIREMENTS

(IEEE SRS Standard Format – Detailed & Structured)

This section defines **what the system must do**.

We will divide it into:

1. User Management
2. Medicine Verification Module
3. RAG & Search Behavior
4. Symptom Remedy Module
5. Admin (Future Scalable)
6. System-Level Functional Behavior

🔹 4.1 User Management Requirements

FR-1: User Registration

The system shall allow users to register using:

- Name
- Email
- Password

Conditions:

- Email must be unique.
- Password must be securely stored (hashed).
- System shall validate email format.

FR-2: User Login

The system shall:

- Authenticate users using email and password.
- Generate JWT token upon successful login.
- Deny access if credentials are invalid.

FR-3: Session Management

The system shall:

- Validate JWT on each protected request.
- Allow logout functionality.
- Expire tokens after defined time.

🔹 **4.2 Medicine Verification Module Requirements**

FR-4: Multi-Modal Input Support

The system shall accept medicine input through:

- Text input (medicine name)
- Image upload (packaging)
- Barcode scanning

FR-5: OCR Processing

If image is uploaded:

- The system shall extract text using OCR.
- The extracted text shall be cleaned and processed.
- Relevant fields (MFG, EXP, MRP) shall be detected using pattern extraction.

FR-6: Barcode Processing

If barcode is provided:

- The system shall decode barcode.
- The decoded value shall be matched against stored medicine barcodes.
- If found, retrieve associated medicine record.

FR-7: RAG-Based Medicine Search

For text-based input:

- The system shall convert input into embedding.
- The system shall perform FAISS similarity search.
- The system shall retrieve top-K closest medicine matches.
- The system shall return similarity score.

Performance Target:

- Search response under 5 seconds (ideal goal).

FR-8: Expiry Verification

The system shall:

- Extract manufacturing and expiry dates.
- Compare expiry date with current date.
- Mark medicine as:
    - Valid
    - Expired

FR-9: Legality Verification

The system shall:

- Retrieve legality status from database.
- Classify medicine as:
    - Legal
    - Banned
    - Restricted
    - Under Review

FR-10: Authenticity Classification

The system shall determine final status:

- Authentic
- Fake
- Expired
- Illegal
- Not Found

Based on:

- Similarity score
- Barcode match
- Expiry status
- Legality status

FR-11: AI Summary Generation

The system shall:

- Generate medicine summary using local LLM.
- Provide:
    - Purpose
    - Usage
    - Basic precautions
- Avoid medical prescription advice.

FR-12: Verification History Storage

The system shall:

- Store verification attempts.
- Store timestamp.
- Store result classification.
- Allow user to view previous history.

🔹 4.3 Symptom-Based Home Remedy Module

FR-13: Symptom Input

The system shall allow users to input free-text symptoms.

Example:
“I have cold and sore throat.”

FR-14: Home Remedy Generation

The system shall:

- Use local LLM to generate safe home remedies.
- Include proportion details.
- Provide precaution notes.
- Avoid medical diagnosis.

FR-15: Mandatory Disclaimer

The system shall always display:

“This information is for educational purposes only and not a substitute for professional medical advice.”

FR-16: Safety Constraints

The system shall NOT:

- Recommend prescription medicines.
- Provide dosage instructions for drugs.
- Claim medical diagnosis.
- Suggest emergency medical treatment.

FR-17: Optional Symptom Logging

If enabled:

- The system shall store symptom input.
- The system shall store generated response.

If disabled:

- No data shall be stored.

🔹 4.4 System-Level Functional Requirements

FR-18: Microservice Communication

The system shall:

- Use REST-based communication between services.
- Handle service failure gracefully.

FR-19: FAISS Persistence

The system shall:

- Persist FAISS index across restarts.
- Allow re-indexing when dataset updates.

FR-20: Docker Deployment

The system shall:

- Run all services inside Docker containers.
- Support development via Docker Compose.
- Allow scalable cloud deployment.

🔹 4.5 Error Handling Requirements

The system shall:

- Return meaningful error messages.
- Handle:
  - Invalid barcode
  - Image unreadable
  - Medicine not found
  - LLM timeout
  - Service unavailable

📘 SECTION 5 – NON-FUNCTIONAL REQUIREMENTS

(IEEE SRS Standard – Quality Attributes of the System)

This section defines **how well** the system must perform.

🔹 5.1 Performance Requirements

NFR-1: Response Time

- Medicine search (RAG + FAISS) shall return results within **5 seconds (target)** under normal load.
- Barcode lookup shall return results within **2 seconds**.
- Symptom remedy generation shall return response within **10 seconds** (depending on LLM size).

NFR-2: Concurrent Users

The system shall support:

- Minimum 100 concurrent users (initial deployment target).
- Scalable to 1000+ users through horizontal scaling.

NFR-3: FAISS Search Performance

- Vector similarity search shall not exceed predefined latency.
- Persistent FAISS index must prevent re-indexing delays.

🔹 **5.2 Scalability Requirements**

NFR-4: Microservice Scalability

Each microservice shall:

- Be independently deployable.
- Be independently scalable.
- Be containerized using Docker.

NFR-5: Database Scalability

PostgreSQL shall:

- Support 50,000+ medicine records.
- Support indexing for fast retrieval.
- Allow future migration to managed cloud DB.

🔹 **5.3 Security Requirements**

NFR-6: Authentication Security

- All protected endpoints must require JWT authentication.
- Passwords must be hashed.
- No plain-text password storage.

NFR-7: Data Transmission Security

- All APIs must run over HTTPS in production.
- Inter-service communication secured via Docker internal network.

NFR-8: Input Validation

The system shall:

- Sanitize user inputs.
- Prevent SQL injection.
- Prevent malicious file uploads.

NFR-9: Role-Based Access Control (Future Scalable)

The system shall support roles:

- User
- Admin
- Pharmacy (future extension)

🔹 5.4 Reliability Requirements

NFR-10: Fault Isolation

If one microservice fails:

- Other services shall continue functioning.
- Error message shall be returned gracefully.

NFR-11: Data Integrity

- Foreign key constraints must ensure relational consistency.
- ENUM types prevent invalid status entries.

🔹 5.5 Availability Requirements

NFR-12: System Uptime

Target:

- 99% uptime in cloud deployment.

NFR-13: Backup Strategy

- Database backups must be scheduled.
- FAISS index should be backed up with persistent volume.

🔹 5.6 Usability Requirements

NFR-14: UI Responsiveness

- Web interface must be responsive.
- Mobile interface must support Android devices.

NFR-15: User Experience

- Clear result classification (color-coded).
- Clear disclaimers.
- Simple and intuitive interface.

🔹 5.7 Maintainability Requirements

NFR-16: Modular Code Structure

- Each service must have separate codebase.

- Clear API documentation.

NFR-17: Embedding Model Upgradability

The system shall:

- Allow re-generation of embeddings.
- Allow LLM model change without system redesign.

🔹 5.8 Compliance & Ethical Requirements

NFR-18: Medical Disclaimer

The system must:

- Clearly state that it is not a substitute for professional medical advice.

NFR-19: No Medical Diagnosis

The symptom module must:

- Avoid diagnosis claims.
- Avoid prescription guidance.

NFR-20: Data Privacy

- User data shall not be shared externally.
- Local LLM ensures no cloud symptom data transmission.

📘 SECTION 6 – SYSTEM CONSTRAINTS

This section defines the **limitations and boundaries** within which MedGuard AI must operate.

Constraints are important because they explain:

- Technical limits
- Resource limits
- Legal limits
- Design decisions
- Deployment restrictions

🔹 6.1 Technical Constraints

SC-1: Technology Stack Constraint

The system must use:

- Django REST Framework (API layer)
- PostgreSQL (Relational Database)
- FAISS (Vector search engine)
- Ollama (Local LLM)

- Docker (Containerization)

These technologies are fixed for Phase-II implementation.

SC-2: Local LLM Resource Constraint

Since Ollama runs locally:

- System performance depends on:
  - Available RAM
  - CPU cores
  - GPU availability (if used)
- Large LLM models may cause:
  - Increased latency
  - High memory consumption

Therefore:
Model size must be selected based on system capability.

SC-3: FAISS Memory Constraint

FAISS stores embeddings in memory for fast retrieval.

Constraints:

- Large dataset increases RAM usage.
- Index rebuild requires time if dataset updates.
- Performance depends on embedding dimension and index type.

SC-4: Docker Environment Constraint

All services must run inside Docker containers.

This introduces:

- Network configuration dependency
- Volume management dependency
- Container orchestration management

🔹 **6.2 Regulatory & Ethical Constraints**

SC-5: Medical Advice Limitation

The system:

- Must not diagnose diseases.
- Must not prescribe medicines.
- Must include medical disclaimer in symptom module.
- Must restrict output to safe informational remedies.

SC-6: Legal Medicine Status Accuracy

Legality information:

- Depends on dataset accuracy.
- May not reflect real-time government updates.
- Requires periodic dataset updates.

🔹 6.3 Data Constraints

SC-7: Dataset Dependency

Medicine verification accuracy depends on:

- Completeness of dataset
- Correct barcode mapping
- Updated legality status

If medicine is not in dataset:
System may classify as “Not Found”.

SC-8: OCR Accuracy Limitation

OCR performance depends on:

- Image clarity
- Lighting conditions
- Packaging quality
- Language of text

System cannot guarantee 100% OCR accuracy.

SC-9: Similarity Threshold Constraint

RAG-based similarity depends on:

- Embedding quality
- Threshold selection
- Semantic match performance

Low similarity threshold may cause:

- False positives
- Incorrect authenticity classification

🔹 6.4 Deployment Constraints

SC-10: Cloud Infrastructure Dependency

Deployment depends on:

- AWS / DigitalOcean / Cloud provider

- Server configuration
- Network bandwidth
- Storage availability

SC-11: Android-Only Mobile Constraint (Initial Phase)

Mobile app (Flutter):

- Initially targeted for Android only.
- iOS deployment may require additional configuration.

🔹 **6.5 Security Constraints**

SC-12: JWT Token Expiry

Authentication sessions depend on:

- Token expiration policy.
- Token refresh mechanism.

Expired tokens require re-login.

🔹 **6.6 Performance Constraints**

SC-13: LLM Response Latency

Symptom module latency depends on:

- Selected LLM model
- Hardware availability
- Concurrent request load

SC-14: Concurrent Request Handling

System performance may degrade if:

- High concurrent users
- Heavy FAISS queries
- Multiple simultaneous LLM generations

Requires horizontal scaling in production.

📘 SECTION 7 – ASSUMPTIONS, DEPENDENCIES, RISK ANALYSIS & FUTURE ENHANCEMENTS

This final section defines:

- Operational assumptions
- System dependencies
- Potential risks
- Future scalability roadmap

🔹 7.1 Assumptions

These are conditions assumed to be true for proper system functioning.

A-1: Dataset Accuracy Assumption

It is assumed that:

- The medicine dataset is accurate.
- Legality information is correctly labeled.
- Barcode mappings are valid.

System accuracy directly depends on dataset quality.

A-2: User Input Quality Assumption

It is assumed that:

- Users provide clear images.
- Barcode images are readable.
- Text input is meaningful and not malicious.

OCR and RAG accuracy depend on input quality.

A-3: Hardware Availability Assumption

It is assumed that:

- The deployment server has sufficient RAM.
- CPU resources are adequate for FAISS and LLM.
- Docker containers are properly configured.

A-4: Regulatory Stability Assumption

It is assumed that:

- Medicine legality data does not change frequently.
- Updates to banned/restricted medicines will be periodically maintained.

🔹 7.2 Dependencies

These are external systems or components MedGuard AI depends on.

D-1: PostgreSQL Dependency

The system depends on PostgreSQL for:

- Structured medicine data
- User authentication data
- History storage

D-2: FAISS Dependency

Medicine search depends on:

- FAISS vector index
- Embedding model consistency
- Persistent volume storage

D-3: Ollama Dependency

LLM-based modules depend on:

- Ollama runtime availability
- Selected local LLM model
- System memory allocation

If Ollama service fails:

- Symptom module becomes unavailable.

D-4: OCR Service Dependency

Image processing depends on:

- OCR engine accuracy
- External or integrated OCR library functionality

D-5: Docker Environment Dependency

All services depend on:

- Proper container orchestration
- Docker network configuration
- Volume mounting stability

🔹 **7.3 Risk Analysis**

This section identifies potential risks and mitigation strategies.

R-1: Incorrect Authenticity Classification

Risk:

- Similar medicine names may produce incorrect FAISS match.

Mitigation:

- Use similarity threshold tuning.
- Combine barcode + text verification.
- Implement confidence scoring.

R-2: Outdated Legality Data

Risk:

- Medicine status may change after dataset update.

Mitigation:

- Periodic dataset review.
- Future integration with government drug APIs.

R-3: LLM Hallucination Risk

Risk:

- LLM may generate unsafe or exaggerated remedies.

Mitigation:

- Strict system prompt.
- Constrained output format.
- Mandatory disclaimer.

R-4: Performance Degradation

Risk:

- High concurrent load.
- Large embedding dataset.
- Heavy LLM usage.

Mitigation:

- Horizontal scaling.
- Load balancing.
- Efficient embedding dimension selection.

R-5: Security Breach Risk

Risk:

- Unauthorized API access.
- Token misuse.

Mitigation:

- JWT expiration.
- HTTPS deployment.
- Input validation.

🔹 7.4 Future Enhancements

MedGuard AI is designed for long-term scalability.

F-1: Government API Integration

- Direct integration with official Indian drug databases.
- Real-time legality updates.

F-2: Pharmacy Dashboard

- Bulk verification mode.
- Medicine stock compliance checks.
- Analytics dashboard.

F-3: Counterfeit Reporting System

- Allow users to report suspicious medicines.
- Generate case reports.
- Notify regulatory authorities (future concept).

F-4: Multi-Language Support

- Hindi
- Marathi
- Regional language interface
- Multilingual OCR support

F-5: Confidence Score Visualization

- Display AI confidence level.
- Improve user trust.

F-6: Advanced AI Health Assistant

- Upgrade symptom module.
- Add conversational memory.
- Risk-level classification (low / moderate / high).

F-7: On-Device OCR & Embedding

- Reduce cloud dependency.
- Improve privacy.
- Edge-device AI deployment.

F-8: iOS Mobile Application

- Expand Flutter support to iOS.
- App Store deployment.

🔥 FINAL SYSTEM SUMMARY

MedGuard AI is:

- A microservice-based
- Dockerized
- RAG-powered
- FAISS-accelerated
- Local-LLM-enabled
- Medicine verification platform

It ensures:

- Authenticity validation
- Expiry detection
- Legal compliance awareness
- Safe home remedy assistance
- Privacy-preserving AI architecture